Homework 4 Solutions

BEE 4850/5850

Overview

Instructions

The goal of this homework assignment is to practice model evaluation and using cross-validation and information criteria to distinguish between models.

- Problem 1 asks you to fit a model for cherry blossom bloom dates and use cross-validation to draw conclusions about generalizability.
- Problem 2 asks you to compare several linear regression models for environmental influences on non-accidental death data.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

The following packages are included in the environment (to help you find other similar packages in other languages). The code below loads these packages for use in the subsequent notebook (the desired functionality for each package is commented next to the package).

```
using Random # random number generation and seed-setting
using DataFrames # tabular data structure
using DataFramesMeta # API which can simplify chains of DataFrames
 ↪  transformations
using CSV# reads/writes .csv files
using Distributions # interface to work with probability distributions
using Plots # plotting library
using StatsBase # statistical quantities like mean, median, etc
using StatsPlots # some additional statistical plotting tools
using Optim
using LaTeXStrings
using Dates

Random.seed!(1)
```

Problems

Problem 1

Problem 1.1

Let's load the data from 'data/HadCRUT5.1Analysis_gl.txt' and 'data/kyoto.csv' and merge the two datasets along common years.

```
temp_dat = CSV.read(joinpath("data", "HadCRUT5.1Analysis_gl.txt"),
 ↪  DataFrame, delim=" ", ignorerepeated=true, header=false)          ①
temp_dat = temp_dat[1:2:nrow(temp_dat), [1, 4]]                       ②
rename!(temp_dat, ["Year", "March"])

kyoto_dat = CSV.read(joinpath("data", "kyoto.csv"), DataFrame,
 ↪  missingstring="NA")
kyoto_dat = kyoto_dat[:, 1:2]
rename!(kyoto_dat, [:Year, :DOY])                                     ③

dat = innerjoin(kyoto_dat, temp_dat, on = :Year)                      ④
dat  = dat[completecases(dat), :]                                     ⑤
disallowmissing!(dat)
```

① The data is space-delimited and the number of spaces between columns differs in the temperature (odd) and number of station (even) rows, so we use `ignorerepeated=true`

to cause the parser to not interpret multiple consecutive spaces as delimiting multiple columns.

② We need to only keep the odd rows (which are the temperatures) and the year/March anomaly columns.

③ The periods in the column names in `kyoto.csv` are annoying for later `DataFrame` operations, so we'll just rename the columns to get rid of them.

④ An `innerjoin` is a type of database join that merges two datasets based on the entries of a common ID column which are present in both datasets.

⑤ There are still some years with `missing` DOYs, so we use `completecases` to remove those and `disallowmissing!` to change the column datatype.

```
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 2. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 4. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 6. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 8. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 10. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 12. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 14. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 16. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 18. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 20. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 22. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 24. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 26. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 28. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 30. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 32. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 34. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 36. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 38. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 40. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 42. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 44. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 46. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 48. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 50. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 52. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 54. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 56. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 58. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 60. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 62. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 64. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 66. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 68. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 70. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 72. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 74. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 76. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 78. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 80. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 82. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 84. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 86. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 88. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 90. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 92. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 94. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 96. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 98. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 100. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 102. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 104. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 106. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 108. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 110. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 112. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 114. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 116. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 118. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 120. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 122. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 124. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 126. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 128. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 130. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 132. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 134. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 136. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 138. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 140. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 142. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 144. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 146. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 148. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 150. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 152. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 154. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 156. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 158. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 160. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 162. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 164. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 166. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 168. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 170. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 172. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 174. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 176. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 178. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 180. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 182. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 184. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 186. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 188. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 190. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 192. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 194. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 196. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 198. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 200. Filling remaining columns with `missing`
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1 warning: only found 13 / 14 columns around data
row: 202. Filling remaining columns with `missing`
```

```
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:592
┌ Warning: thread = 1: too many warnings, silencing any further warnings
└ @ CSV ~/.julia/packages/CSV/LiiJM/src/file.jl:597
```

	Year	DOY	March
	Int64	Int64	Float64
1	1850	109	-0.627
2	1851	102	-0.641
3	1852	113	-0.567
4	1853	102	-0.273
5	1854	102	-0.247
6	1855	101	-0.345
7	1856	111	-0.371
8	1857	108	-0.488
9	1858	104	-0.566
10	1859	110	-0.322
11	1860	113	-0.789
12	1861	100	-0.422
13	1862	117	-0.538
14	1863	118	-0.414
15	1864	105	-0.531
16	1865	97	-0.717
17	1866	102	-0.748
18	1867	101	-0.712
19	1868	104	-0.332
20	1869	105	-0.651
21	1870	99	-0.495
22	1871	104	-0.171
23	1873	104	-0.39
24	1874	107	-0.664
…	…	…	…

Now let's fit the model.

```
function cherry_loglik(p, doy, temp)
    b0, b1, s = p
    m = b0 .+ b1 * temp
    ll = sum(logpdf.(Normal.(m, s), doy))
    return ll
end

lb = [-200., -100., 0.01]
ub = [200., 100., 10.]
```

```
p0 = [100., 5., 5.]
optim_out = optimize(p -> -cherry_loglik(p, dat.DOY, dat.March), lb, ub,
 ↪  p0)
p_mle = optim_out.minimizer
@show p_mle;
```

```
p_mle = [100.6710269629213, -7.84224390609888, 4.717152358910784]
```

This suggests that every degree of increased average March temperature makes the cherry blossoms flower around 8 days earlier.

Problem 1.2

We can get the log-loss metric directly from the optimization, as the negative log-likelihood was the optimization objective.

```
ll = optim_out.minimum
@show ll;
```

```
ll = 478.19315773357795
```

For the mean-square error, we need to compute the predictions.

```
pred = p_mle[1] .+ p_mle[2] * dat.March
mse = mean((pred .- dat.DOY).^2)
@show mse;
```

```
mse = 22.251526373148707
```

Problem 1.3

To conduct a 5-fold cross-validation, we need to split the data and then repeat the earlier workflow and store the metrics. Since we're modeling this using a linear regression, which assumes that the observations are independent, we can do an independent split by random partitioning of the data rows. Let's write a function to do this.

```
function cherry_cv(dat, nfolds = 5)
    # split the data
    labels = repeat(1:nfolds, outer = Int(floor(nrow(dat) / nfolds)))  ①
    labels = [labels; 1]
    fold_labels = shuffle(labels) # randomly permite the labels
```

```
    # create storage for error metrics
    cv_ll = zeros(nfolds)
    cv_mse = zeros(nfolds)
    # set optimization parameters
    lb = [-200., -100., 0.01]
    ub = [200., 100., 10.]
    p0 = [100., 5., 5.]
    for fold in 1:nfolds
        test_idx = findall(fold_labels .== fold)
        train_idx = setdiff(1:nrow(dat), test_idx)
        # fit model to the fold
        optim_out = optimize(p -> -cherry_loglik(p, dat[train_idx,
↪   :DOY], dat[train_idx, :March]), lb, ub, p0)
        p_mle = optim_out.minimizer
        pred = p_mle[1] .+ p_mle[2] * dat[test_idx, :March]
        cv_ll[fold] = -mean(logpdf.(Normal.(pred, p_mle[3]),
↪   dat[test_idx, :DOY]))                                          ②
        cv_mse[fold] = mean((pred .- dat[test_idx, :DOY]).^2)
    end
    return cv_ll, cv_mse
end
cv_ll, cv_mse = cherry_cv(dat, 5)
```

① The number of data points is not divisible evenly by 5, so we just create as close to an even set of splits as we can and then add the last data point to one of the remaining folds (in this case the first one).

② We get the mean log score here because the number of data points differs between the CV folds and the full-data.

```
([3.0150202460994526, 3.0283992864632503, 2.8857968018404456,
3.0286095541526157, 2.9459127505005456], [24.20962065059353,
24.765915255574352, 18.289382346763833, 24.772091567081112,
21.155985453526426])
```

Now let's compare to the total-data/in-sample tests.

```
@show ll / nrow(dat);
@show mean(cv_ll);
```

```
ll / nrow(dat) = 2.9701438368545214
mean(cv_ll) = 2.980747727811262
```

```
@show mse;
@show mean(cv_mse);
```

```
mse = 22.251526373148707
mean(cv_mse) = 22.63859905470785
```

We can see that in general, the model's in-sample performance is not too different than its average CV performance, suggesting that there is no systematic overfitting occurring.

Problem 1.4

Since there is no evidence of overfitting from the cross-validation test, we might conclude that the model generalizes well.

Problem 2

Problem 2.1

Loading and plotting the deaths data from `data/chicago.csv`:

```
chicago_dat = CSV.read(joinpath("data", "chicago.csv"), DataFrame,
 ↪  delim=",", missingstring="NA", header=true)
day_zero = Date("1993-12-31")
chicago_dat.Date = day_zero .+ Day.(chicago_dat.time .+ 0.5)
```

```
UndefVarError: UndefVarError(:Date, Main.Notebook)
UndefVarError: `Date` not defined in `Main.Notebook`
Suggestion: check for spelling errors or missing imports.
Hint: a global variable of this name may be made accessible by importing
 ↪  Dates in the current active module Main
Stacktrace:
 [1] top-level scope
   @ ~/Teaching/BEE4850/solutions/hw04/hw04.qmd:216
```

Figure 1

To ensure that we use the same observations for the purposes of model comparison, let's remove the cases corresponding to missing PM2.5 data.

```
chicago_dat = chicago_dat[.!(ismissing).(chicago_dat.pm25median), :]

p_deaths = plot(chicago_dat.Date, chicago_dat.death, lw=2,
 ↪  xlabel="Date", ylabel="Non-Accidental Deaths)", label="Data",
 ↪  color=:gray)
```

```
ArgumentError: ArgumentError("column name \"Date\" not found in the data
 ↪  frame; existing most similar names are: \"death\"")
ArgumentError: column name "Date" not found in the data frame; existing
 ↪  most similar names are: "death"
Stacktrace:
 [1] lookupname
   @ ~/.julia/packages/DataFrames/b4w9K/src/other/index.jl:434 [inlined]
 [2] getindex
   @ ~/.julia/packages/DataFrames/b4w9K/src/other/index.jl:440 [inlined]
 [3] getindex(df::DataFrame, ::typeof(!), col_ind::Symbol)
   @ DataFrames
 ↪  ~/.julia/packages/DataFrames/b4w9K/src/dataframe/dataframe.jl:557
 [4] getproperty(df::DataFrame, col_ind::Symbol)
   @ DataFrames
 ↪  ~/.julia/packages/DataFrames/b4w9K/src/abstractdataframe/abstractdataframe.jl:448
 [5] top-level scope
   @ ~/Teaching/BEE4850/solutions/hw04/hw04.qmd:231
```

Figure 2

We could fit the regression models in a few different ways (in Julia, we could also use the `GLM.jl` package), but for consistency's sake we'll do this with numerical optimization. First, let's fit a model for the impact of time.

```
# general log-likelihood function for a linear regression with covariate
 ↪  x
function timereg_loglik(p, y, t)
    μ₀, μ₁, σ = p
    μ = μ₀ .+ μ₁ * t
    ll = sum(logpdf.(Normal.(μ, σ), y))
    return ll
end

lb = [0.0, -100.0, 0.1]
ub = [200.0, 100.0, 20.0]
x₀ = [100.0, 0.0, 5.0]

optim_out = Optim.optimize(p -> -timereg_loglik(p, chicago_dat.death,
 ↪  chicago_dat.time), lb, ub, x₀)
ll_time = -optim_out.minimum                                              ①
θ_time = optim_out.minimizer
@show round.(θ_time; digits=1);
```

① Since we minimized the *negative* log-likelihood, to get the log-likelihood we need to take the negative of the minimum value again.

```
round.(θ_time; digits = 1) = [101.7, 0.0, 14.1]
```

Plotting this regression over the data:

```
plot!(p_deaths, chicago_dat.Date, θ_time[1] .+ θ_time[2] *
 ↪  chicago_dat.time, lw=2, linestyle=:dash, color=:red, label="Time
 ↪  Regression")
```

```
UndefVarError: UndefVarError(:p_deaths, Main.Notebook)
UndefVarError: `p_deaths` not defined in `Main.Notebook`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ ~/Teaching/BEE4850/solutions/hw04/hw04.qmd:268
```

Figure 3

We can see that even though the time coefficient is close to zero, suggesting a minimal time-dependent trend, over the course of the dataset there is a slight increase over this time (seen in Figure 3).

Problem 2.2

Now let's look at the impact of temperature.

```
# general log-likelihood function for a linear regression with covariate
↪  x
function tempreg_loglik(p, y, temp)
    μ₀, μ₁, σ = p
    μ = μ₀ .+ μ₁ * temp
    ll = sum(logpdf.(Normal.(μ, σ), y))
    return ll
end

lb = [0.0, -100.0, 0.1]
ub = [200.0, 100.0, 20.0]
x₀ = [100.0, 0.0, 5.0]

optim_out = Optim.optimize(p -> -tempreg_loglik(p, chicago_dat.death,
↪  chicago_dat.tmpd), lb, ub, x₀)
ll_temp = -optim_out.minimum
θ_temp = optim_out.minimizer
@show round.(θ_temp; digits=1);
```

```
round.(θ_temp; digits = 1) = [129.2, -0.4, 12.4]
```

Adding this regression line to the plot:

Figure 4 shows that the temperature regression results in some oscillations due to the seasonal impact of temperature, which visually appear to align with the oscillations in the deaths data.

Problem 2.3

Our last model adds PM2.5 concentrations to the previous temperature regression. Here we need to drop the entries corresponding to missing data.

```
# general log-likelihood function for a linear regression with covariate
↪  x
function multireg_loglik(p, y, temp, pm25)
```

```
plot!(p_deaths, chicago_dat.Date, θ_temp[1] .+ θ_temp[2] *
 ↪  chicago_dat.tmpd, lw=2, linestyle=:dash, color=:blue,
 ↪  label="Temperature Regression")
```

```
UndefVarError: UndefVarError(:p_deaths, Main.Notebook)
UndefVarError: `p_deaths` not defined in `Main.Notebook`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ ~/Teaching/BEE4850/solutions/hw04/hw04.qmd:308
```

Figure 4

```
    μ₀, μ₁, μ₂, σ = p
    μ = μ₀ .+ μ₁ * temp .+ μ₂ * pm25
    ll = sum(logpdf.(Normal.(μ, σ), y))
    return ll
end

lb = [0.0, -100.0, -100.0, 0.1]
ub = [200.0, 100.0, 100.0, 20.0]
x₀ = [100.0, 0.0, 0.0, 5.0]

optim_out = Optim.optimize(p -> -multireg_loglik(p, chicago_dat.death,
 ↪  chicago_dat.tmpd, chicago_dat.pm25median), lb, ub, x₀)
ll_multi = -optim_out.minimum
θ_multi = optim_out.minimizer
@show round.(θ_multi; digits=1);
```

```
round.(θ_multi; digits = 1) = [129.3, -0.4, 0.2, 12.3]
```

We add this regression to the plot:

Problem 2.4

Now we want to conduct a 5-fold cross-validation to compare each model's performance. As set up, we don't need to account for any particular structure, as we aren't accounting for any underlying seasonal effects that aren't captured by the covariates.

We will write a function to conduct these for each of our models.

```
plot!(p_deaths, chicago_dat.Date, θ_multi[1] .+ θ_multi[2] *
↪  chicago_dat.tmpd .+ θ_multi[3] * chicago_dat.pm25median, lw=2,
↪  linestyle=:dash, color=:violet, label="Temperature and PM2.5
↪  Regression")
```

```
UndefVarError: UndefVarError(:p_deaths, Main.Notebook)
UndefVarError: `p_deaths` not defined in `Main.Notebook`
Suggestion: check for spelling errors or missing imports.
Stacktrace:
 [1] top-level scope
   @ ~/Teaching/BEE4850/solutions/hw04/hw04.qmd:347
```

Figure 5

```
function time_cv(dat, nfolds=5)
    # split the data
    labels = repeat(1:nfolds, outer = Int(floor(nrow(dat) / nfolds)))  ①
    labels = [labels; 1; 2]
    fold_labels = shuffle(labels) # randomly permite the labels
    # create storage for error metrics
    cv_ll = zeros(nfolds)
    cv_mse = zeros(nfolds)
    # set optimization parameters
    lb = [0.0, -100.0, 0.1]
    ub = [200.0, 100.0, 20.0]
    p0 = [100.0, 0.0, 5.0]
    for fold in 1:nfolds
        test_idx = findall(fold_labels .== fold)
        train_idx = setdiff(1:nrow(dat), test_idx)
        # fit model to the fold
        optim_out = optimize(p -> -timereg_loglik(p, dat[train_idx,
↪  :death], dat[train_idx, :time]), lb, ub, p0)
        p_mle = optim_out.minimizer
        pred = p_mle[1] .+ p_mle[2] * dat[test_idx, :time]
        cv_ll[fold] = -mean(logpdf.(Normal.(pred, p_mle[3]),
↪  dat[test_idx, :death]))                                          ②
        cv_mse[fold] = mean((pred .- dat[test_idx, :death]).^2)
    end
    return cv_ll, cv_mse
end
```

```
function temp_cv(dat, nfolds=5)
    # split the data
    labels = repeat(1:nfolds, outer = Int(floor(nrow(dat) / nfolds)))  ①
    labels = [labels; 1; 2]
    fold_labels = shuffle(labels) # randomly permite the labels
    # create storage for error metrics
    cv_ll = zeros(nfolds)
    cv_mse = zeros(nfolds)
    # set optimization parameters
    lb = [0.0, -100.0, 0.1]
    ub = [200.0, 100.0, 20.0]
    p0 = [100.0, 0.0, 5.0]
    for fold in 1:nfolds
        test_idx = findall(fold_labels .== fold)
        train_idx = setdiff(1:nrow(dat), test_idx)
        # fit model to the fold
        optim_out = optimize(p -> -tempreg_loglik(p, dat[train_idx,
↪   :death], dat[train_idx, :tmpd]), lb, ub, p0)
        p_mle = optim_out.minimizer
        pred = p_mle[1] .+ p_mle[2] * dat[test_idx, :tmpd]
        cv_ll[fold] = -mean(logpdf.(Normal.(pred, p_mle[3]),
↪   dat[test_idx, :death]))                                          ②
        cv_mse[fold] = mean((pred .- dat[test_idx, :death]).^2)
    end
    return cv_ll, cv_mse
end

function temp_pm_cv(dat, nfolds=5)
    # split the data
    labels = repeat(1:nfolds, outer = Int(floor(nrow(dat) / nfolds)))  ①
    labels = [labels; 1; 2]
    fold_labels = shuffle(labels) # randomly permite the labels
    # create storage for error metrics
    cv_ll = zeros(nfolds)
    cv_mse = zeros(nfolds)
    # set optimization parameters
    lb = [0.0, -100.0, -100.0, 0.1]
    ub = [200.0, 100.0, 100.0, 20.0]
    p0 = [100.0, 0.0, 0.0, 5.0]

    for fold in 1:nfolds
        test_idx = findall(fold_labels .== fold)
        train_idx = setdiff(1:nrow(dat), test_idx)
```

```
        # fit model to the fold
        optim_out = optimize(p -> -multireg_loglik(p, dat[train_idx,
↪  :death], dat[train_idx, :tmpd], dat[train_idx, :pm25median]), lb,
↪  ub, p0)
        p_mle = optim_out.minimizer
        pred = p_mle[1] .+ p_mle[2] * dat[test_idx, :tmpd] .+ p_mle[3] *
↪  dat[test_idx, :pm25median]
        cv_ll[fold] = -mean(logpdf.(Normal.(pred, p_mle[4]),
↪  dat[test_idx, :death]))                                              ②
        cv_mse[fold] = mean((pred .- dat[test_idx, :death]).^2)
    end
    return cv_ll, cv_mse

end
```

```
temp_pm_cv (generic function with 2 methods)
```

Now let's compare the results for each model.

```
time_cv_ll, time_cv_mse = time_cv(chicago_dat, 5)
temp_cv_ll, temp_cv_mse = temp_cv(chicago_dat, 5)
multi_cv_ll, multi_cv_mse = temp_pm_cv(chicago_dat, 5)

cv_models = ["Time", "Temperature", "Temperature + PM 2.5"]
cv_ll_all = [mean(time_cv_ll), mean(temp_cv_ll), mean(multi_cv_ll)]
cv_mse_all = [mean(time_cv_mse), mean(temp_cv_mse), mean(multi_cv_mse)]

cv_results = DataFrame(Model=cv_models, LogScore=cv_ll_all,
↪  MSE=cv_mse_all)
```

	Model	LogScore	MSE
	String	Float64	Float64
1	Time	4.07372	201.236
2	Temperature	3.94227	155.441
3	Temperature + PM 2.5	3.93076	151.581

By these scores, the temperature + PM 2.5 model scores best for both the log score and the MSE, so we might conclude that it would generalize the best out-of-sample. The temperature-only model performs similarly, and the time model, as we saw in Problem 2.1, does not perform particularly well.

Problem 2.5

To compute the AIC values we will use the log-likelihoods we computed when we fit each model.

```
aic_time = -2 * (ll_time - 3)
aic_temp = -2 * (ll_temp - 3)
aic_multi = -2 * (ll_multi - 4)
aic_df = DataFrame(Models=["Time", "Temperature", "Temperature and
 ↪  PM2.5"], AIC=round.([aic_time, aic_temp, aic_multi]; digits=0))
```

	Models	AIC
	String	Float64
1	Time	5918.0
2	Temperature	5729.0
3	Temperature and PM2.5	5714.0

We can see that the Temperature and PM2.5 model minimizes AIC, and the ΔAIC for the temperature-only model is around 15. These differences suggest that there is relatively strong evidence for the influence of both temperature and PM2.5 compared to a temperature-only model, as the predictive skill outweighs the potential overfitting from adding the PM2.5 term. This is similar in direction to what we saw from the cross-validation as well.

When comparing this quantitative result to the plots, drawing this conclusion from Figure 5 itself is challenging, even if we might be able to squint and see that the fluctuations from the PM2.5 model align slightly better with the temperature-only model.